Date and Time Functions

PAGE NO.....
DATE / /

~~database: date_function~~

Setting datetime field values

1. Datetime Fields

→ Datetime fields store both date and time in a single column.

→ They are useful for calculations like time differences or filtering based on specific date and time ranges.

2. STR_TO_DATE functions

→ Converts a string into a datetime value.

→ Require a format string to interpret the input string correctly.

→ Example format: '%Y-%m-%d %h:%i %p' for YYYY-MM-DD HH:MM AM/PM

3. CONCAT Functions

→ Use CONCAT to merge date and time strings.

→ Use STR_TO_DATE to convert the concatenated string into a datetime value.

Small Y.y is for 2 digit.
Large Y.Y is for 4 digit.

PAGE NO.....
DATE / /

market_data_info

Market_date	market_start_time	market_end_time
2023-10-01	08:00 AM	02:00 PM
2023-10-02	09:00 AM	03:00 PM
2023-10-03	10:00 AM	04:00 PM
Date	Varchar(10)	Varchar(10)

-create Datetime Table.

```
Create Table datetime_demo As
(
  Select
    market_date,
    market_start_time,
    market_end_time,
    STR_TO_DATE(CONCAT(market_date, ' ', market_start_time
      , '%Y-%m-%d %h:%i %p')
      As market_start_datetime,

    STR_TO_DATE(CONCAT(market_date, ' ', market_end
      _time), '%Y-%m-%d %h:%i %
      As market_end_datetime
    From market_data_info
);
Select * From datetime_demo; -- Query and Analyze.
```

market_date	market_start_time	market_end_time	market_start_datetime
2023-10-01	08:00 AM	02:00 PM	2023-10-01 08:00:00
2023-10-02	09:00 AM	03:00 PM	2023-10-02 09:00:00
2023-10-03	10:00 AM	04:00 PM	2023-10-03 10:00:00

PAGE NO.....
DATE / /

Explanation of the Query:

1. CONCAT(market_date, ' ', market_start_date);
   -> Combines market_date and market_start_time into a single string.
   -> Example: '2023-10-01 08:00 AM'

2) STR_TO_DATE(....., '%Y-%m-%d %h:%i %p');
   -> Converts the concatenated string into a datetime value.
   -> %Y-%m-%d %h:%i %p specifies the format:
   YYYY-MM-DD HH:MM AM/PM

3) Resulting Datetime Fields.
   -> market_start_datetime and market_end_datetime are stored as proper datetime values.

Why is there an ' ' in the middle?

-> The ' ' (space) in CONCAT(market_date, ' ', market_start_time) is used to separate the date and time in the concatenated string. Without it, the date and time would be merged together making it impossible for the STR_TO_DATE function to interpret the string correctly.

Does CONCAT convert dates into string?

Yes, CONCAT treats all inputs as strings and combines them into a single string. Even if the input is a date or time, CONCAT will convert it to a string before merging.

Why is STR_TO_DATE needed?

The STR_TO_DATE function converts the concatenated string into a datetime value.

market_end_datetime
2023-10-01 14:00:00
2023-10-02 15:00:00
2023-10-03 16:00:00

PAGE NO.....
DATE / /

Extract and Date_part

1) Datetime Extraction:

-> Datetime fields store both date and time, but sometimes you only need specific parts (e.g. year, month, day, hour, minute).

-> Function like Extract, DATE and TIME help retrieve these part.

2) EXTRACT Function:

-> Extract specific parts (e.g. year, month, day, hour, minute) from a datetime field.

-> Syntax:

Extract (part From datetime_field).

-> Example:

Extract (Year From '2023-10-01 08:00:00') returns 2023.

3) DATE and TIME Functions:

-> Date (datetime_field) extracts the date part (e.g. 2023-10-01).

-> Time (datetime_field) extracts the time part (e.g. 08:00:00)

4) Use Cases:

-> Year-over-year comparisons (extract year, month, and day).

-> Grouping data by specific time intervals (e.g. hourly, daily, monthly).

PAGE NO.
DATE / /

~~Table name~~ = datatime_demo1

market_start_datatime	
2023-10-01	08:00:00
2023-10-02	09:30:00
2023-10:03	10:45:00

Datatime.

1) Use EXTRACT to Retrieve Data Parts.

```
Select
    market_start_datatime,
    Extract ( DAY FROM market_start_datatime) As mktsat_day;

Extract ( Month From market_start_datatime) As mktsat_month,
Extract ( Year From market_start_datatime) As mktsat_year,
Extract ( Hour From market_start_datatime) As mktsat_hour,
Extract ( Minute From market_start_datatime) As mktsat_minute
FROM datatime_demo1;
```

output

market_start_datatime	mktsat_day	mktsat_month	mktsat_year	mktsat_hour	mktsat_minute.
2023-10-01 08:00:00	1	10	2023	8	0
2023-10-02 09:30:00	2	10	2023	9	30.
2023-10-03 10:45:00	3	10	2023	10.	43.

2) Use DATE and TIME to Extract Data and Time.

```
Select
    market_start_datatime,
    DATE ( market_start_datatime ) As mktsat_date.
    TIME ( market_start_datatime) As mksat_time
From datatime_demo1;
```

PAGE NO.....
DATE / /

output

market_start_deltatime	mktsat_delta	mktsat_time.
2023-10-01 08:00:00	2023-10-01	08:00:00.
2023-10-02 09:30:00	2023-10-02	09:30:00.
2023-10-03 10:45:00	2023-10-03	10:45:00.

3) Filter Using Extracted Parts.

```
Select *
From
Where
    Extract ( Year From market_start_deltatime) =2023.
    AND Extract( Month From market_start_deltatime ) =10;
```

market_start_deltatime.	
▶ 2023-10-01	08:00:00
2023-10-02	09:30:00
2023-10-03	10:45:00.

dhawalone.com

DATE_ADD and DATE_SUB.

SQL provides powerfull data manipulation functions like DATE_ADD() and DATE_SUB() to dynamically calculate dates and times. These functions allow you to perform date arithmetic on DATE and DATETIME fields, which is crucial for scheduling / forecasting and time-based analysis.

1.) Date Calculations:

-> Datetime values allow for mathematical operations like addition and subtraction, which are not possible with string representations of dates.

-> Functions like DATE_ADD and DATE_SUB are used to perform these operations.

2) DATE_ADD Function:

-> Adds a specified interval (e.g. days, minutes, hours) to a datetime value.

-> Syntax

DATE_ADD (datetime_field, INTERVAL, value unit)

-> Example: DATE_ADD ('2023-10-01 08:00:00', INTERVAL 30 MINUTE).

adds 30 minutes.

3) DATE_SUB Function:

-> Subtracts a specified interval from a datetime value.

-> Syntax

DATE_SUB (datetime_field, INTERVAL value unit)

-> Example: DATE_SUB ('2023-10-01' 08:00:00', INTERVAL 30 DAY).

Subtract 30 days.

PAGE NO.....
DATE / /

4. Negative intervals

→ You can use DATE_ADD with a negative interval to achieve the same result as DATE_SUB

→ Example: DATE_ADD('2023-10-01 08:00:00', INTERVAL -30 DAY) is equivalent to DATE_SUB('2023-10-01 08:00:00', INTERVAL 30 DAY)

datetime_demo2

	market_start_datetime
▶	2023-10-01 08:00:00
	2023-10-02 09:30:00
	2023-10-03 10:45:00

1) Use DATE_ADD to Add Intervals.

-) Add 30 minutes and 30 days to the market_start_datetime field.

```
Select
    market_start_datetime,
    DATE_ADD(market_start_datetime, Interval 30 MINUTE) AS
        mktstart_plus_30min,
    DATE_ADD(market_start_datetime, Interval 30 DAY) As
        mktstart_plus_30days.
```

Output

	market_start_datetime	mktstart_plus_30min	mktstart_plus_30days.
▶	2023-10-01 08:00:00	2023-10-01 08:30:00	2023-10-31 08:00:00
	2023-10-02 09:30:00	2023-10-02 10:00:00	2023-11-01 09:30:00
	2023-10-03 10:45:00	2023-10-03 11:15:00	2023-11-02 10:45:00

PAGE NO.....
DATE / /

DATE DIFF

1. DATE DIFF Functions

-> Calculate the difference between two dates or datetimes values.
-> Returns the difference in days
-> Syntax: DATEDIFF(end_date, start_date)
-> Example: DATEDIFF('2023-10-15', '2023-10-10') return 14.

2. Use Cases:

-> Calculate the number of days between two events (e.g. first and last purchase).
-> Determine if two dates are within a specific range (e.g. 30 days).

3. Nested Queries

-> Use subqueries to calculate intermediate results (e.g. first and last dates).
-> Pass the results to the outer query for further calculations.

datetime_demo3

market	start_datetime
2023-10-01	08:00:00
2023-10-05	09:30:00
2023-10-10	10:45:00

DATETIME

PAGE NO.....
DATE / /

1) Use DATEDIFF to Calculate Date Difference

```
Select
    x. first_market,
    x. last_market,
DATEDIFF (x. last_market, x. first_market) As
                    days_first_to_last
From
    (
     SELECT
          MIN (market_start_datetime) As First_market
          MAX (market_start_datetime) As Last_market
     FROM datetime_demo3
    ) x;
```

Breakdown:

1. Subquery (Inner Query).

-> The subquery calculates the minimum (MIN) and maximum (MAX) values of the market_start_datetime column from the datetime_demo3 table.

-> These values are aliased as first_market (minimum datetime) and last_market (maximum datetime)

-> The entire subquery is aliased as x, which means the results of the subquery are treated as temporary table named 'x'.

-> The subquery will return:

first_market	last_market
2023-10-01 08:00:00	2023-10-10 10:45:00

-> This result is stored in the temporary table 'x'

PAGE NO.....
DATE / /

2) outerquery

-> m.first_market refers to the first_market column in the temporary table m (which contains the minimum datetime value).

-> m.last_market refers to the last_market column in the temporary table m (which contains the maximum datetime value).

-> DATEDIFF (m.last_market, m.first_market) calculate the difference in days between Last_market and first_market.

-> The result is aliased as days_first_to_last.

output

first_market	Last_market	days_first_to_last
2023-10-01 08:00:00	2023-10-10 10:45:00	9.

movie_bookings

booking_id	movie_name	show_datetime	duration_minutes
1	Inception	2025-02-12 18:00:00	150
2	Interstellar	2025-02-12 21:00:00	165
3	The Dark Knight	2025-02-13 17:30:00	152

employees_records

employee_id	name	join_date	last_promotion_date
1	Alice	2020-05-15	2023-08-30
2	Bob	2018-11-02	2022-07-10
3	Charlie	2019-06-25	2024-01-03

Int Primary Key | varchar (100) | DATE | DATE

PAGE NO.....
DATE / /

1) Calculate Employee Tenure and Time Since Last Promotion.

~~name join_date last_promo~~

```
Select
    name,
    join_date,
    last_promotion_date,
    DATEDIFF(CURDATE(), join_date) As tenure_days,
    DATEDIFF(CURDATE(), last_promotion_date) As days_since_promotion
From employee_records;
```

Explanation of Query

1) CURDATE()
-> Returns the current date (e.g. 2023-10-25 if today is October 25, 2023).

2) DATEDIFF(end_time, start_date);
-> Calculates the difference in days between two dates.
-> DATEDIFF(CURDATE(), join_date) calculate the no. of days since the employee joined.
-> DATEDIFF(CURDATE(), last_promotion_date) calculate the no. of days since the last promotion.

output

name	join_date	last_promotion_date	tenure_days	days_since_promotions
Alice	2020-05-13	2023-08-20	1797	545
Bob	2018-11-02	2022-07-10	2297	951
Charlie	2019-06-25	2024-01-05	2062	407

PAGE NO.....
DATE / /

2) Find Employees who have NOT Been Promoted in the last 2 years.

```
Select
    name,
    last_promotion_date,
From employee_records
Where DATEDIFF(CURDATE(), last_promotion_date) > 730;
```

output

name	last_promotion_date
Bob	2022-07-10

3) Find Employees who got Promoted within 30 Days of Joining

```
Select
    name,
    join_date,
    last_promotion_date,
    DATEDIFF(last_promotion_date, join_date) As days_between_joins_promotion
FROM employee_records
where DATEDIFF(last_promotion_date, join_date) <= 30;
```

Here "DATEDIFF(last_promotion_date, join_date) <= 2000 As days_between_join_promotion" has already done, can't we just put "WHERE DATEDIFF(days_between_join_promotion) < = 2000 instead of "where DATEDIFF(last_promotion, join_date) < = 2000".

PAGE NO.....
DATE / /

Why You Can't Use the Alias in the WHERE Clause

→ The WHERE clause is executed before the SELECT clause so the alias days_between_join_promotion is not yet available when the WHERE clause is processed.

output

Note → There is no employee got Promoted within 30 days of Joining.

dhawa dorje ghising

TIMESTAMPDIFF

PAGE NO.....
DATE / /

1) TIMESTAMPDIFF Function:

→ Calculate the difference between two datetime values in a specified interval (e.g. seconds, minutes, hours, days, months, years)

→ Syntax
TIMESTAMPDIFF (unit, start_datetime, end_datetime).

→ Example: TIMESTAMPDIFF(HOUR, '2023-10-01 08:00:00', '2023-10-01 14:00:00') return 6 (hours).

2) Intervals Supported.

→ Second, Minute, Hour, Day, Month, Year

3) Use Cases.

→ Calculate the duration of events (e.g. market hours, meeting durations);

→ Compare datetime values in different units (e.g. hours, minutes).

market_datetime.

market_start_datetime	market_end_datetime
2023-10-01 08:00:00	2023-10-01 14:00:00
2023-10-02 09:30:00	2023-10-02 16:45:00
2023-10-03 10:00:00	2023-10-03 18:30:00

DATETIME DATETIME

PAGE NO.....
DATE ../ /

```
Select
    market_start_datetime,
    market_end_datetime,
TIMESTAMPDIFF(HOUR, market_start_datetime, market_end
    _datetime) As market_duration_hours.
TIMESTAMPDIFF(MINUTE, market_start_datetime, market_end
    _datetime) As market_duration_hours.
From market_datetime;
```

output

market_start_datetime	market_end_datetime	Market duration hours	Market duration mins.
2023-10-01 08:00:00	2023-10-01 14:00:00	6	360
2023-10-02 09:30:00	2023-10-02 16:41:00	7	433
2023-10-03 10:00:00	2023-10-03 18:30:00	8	520

Exploring Multiple Tables Simultaneously

PAGE NO.....
DATE / /

1) Exploring Multiple Tables

→ In relational databases, data is often spread across multiple tables.

→ To analyze relationship between tables, you need to join or filter data based on common fields (e.g. vendor_id, product_id).

2) Aggregating Data:

→ Use GroupBy to summarize data (e.g. total sales, total quantity sold).
→ Use aggregate functions like SUM, COUNT, AVG etc.

3) Filtering Data:

→ Use WHERE to filter rows based on conditions (e.g. specific vendor, product, or customer)
→ Use ORDER BY to sort results for better readability.

4) Use Cases

→ Compare sales data with inventory data.
→ Analyze customer purchase behavior
→ Generate summaries for reporting.

Unacademy Rising

PAGE NO.....
DATE

purchase_id	vendor_id	product_id	market_date	customer_id	quantity	cost_to_customer_per_qty	transaction_time
1	7	4	2023-04-03	12	2.00	4.00	09:13:00
2	7	4	2023-04-03	15	1.50	4.00	10:30:00
3	7	4	2023-04-06	12	5.00	4.00	11:45:00
4	7	4	2023-04-06	12	5.00	4.00	12:15:00
5	7	4	2023-4-06	18	3.00	4.00	13:00:00
6	7	4	2023-04-10	12	4:00	4.00	09:30:00
7	7	4	2023-04-10	22	2.50	4.00	10:43:00

```
Select
    market_date,
    vendor_id,
    product_id,
    SUM(quantity) As quantity_sold,
    SUM(quantity * cost_to_customer_per_qty) As total_sales
From customer_purchases
Where vendor_id = 7 AND product_id = 4
Group by market_date, vendor_id, product_id
Order by market_date;
```

PAGE NO.....
DATE / /

Output

market_date	vendor_id	product_id	quantity_sold	total_sales
2023-04-03	7	4	3.50	14.00
2023-04-06	7	4	13.00	52.00
2023-04-10	7	4	6.50	26.00

** Explanation of the Output.

1) 2023-04-03:
   - Quantity Sold: 2.00 + 1.50 = 3.50
   - Total Sales: (2.00 * 4.00) + (1.50 * 4.00) = 8.00 + 6.00 + 4.00

2.) 2023-04-06:
   - Quantity Sold: 5.00 + 5.00 + 3.00 = 13.00
   - Total Sales: (5.00 * 4.00) + (5.00 * 4.00) + (3.00 * 4.00)
     = 20.00 + 20.00 + 12.00 = 52.00.

3.) 2023-04-10:
   - Quantity Sold: 4.00 + 2.50 = 6.50
   - Total Sales: (4.00 * 4.00) + (2.50 * 4.00) = 16.00 + 10.00 = 26.00.

dhanadorie.com/rising